

Pipeline RAG multi-dominio con scraping concurrente y aislamiento vectorial

Asistente normativo ASFI / Banco Unión · pgvector 768 dims · 672 fragmentos en 4 colecciones · informe con todas las cifras leídas de resultados/

1. Resumen

Se construyó un pipeline de ingesta y consulta RAG sobre normativa boliviana con tres requisitos que se refuerzan entre sí: ingesta concurrente, aislamiento estricto por dominio y coste de embeddings acotado. La decisión de diseño central es que el **aislamiento lo garantiza el motor de base de datos y no el código de aplicación**: la RPC de vecindad recibe `coleccion_id` como filtro obligatorio y es `SECURITY DEFINER`, de modo que una consulta no puede leer otra colección aunque el llamador lo intente.

Resultados principales. 120 consultas de aislamiento con **0 intrusiones**, 0 consultas vacías y 0 filas en colecciones inexistentes; el control negativo devuelve 20/20 + 20/20 + 20/20 filas ajenas, lo que prueba que el test detectaría contaminación si existiera. En Docker limitado a 2 vCPU, speedup de **4.20x** a 8 procesos con **58%** de eficiencia frente al techo de Amdahl. La caché de embeddings resuelve 449 textos sin una sola petición a la API, y la fase de embeddings en la Serie B queda en 0.19 s.

2. Arquitectura y aislamiento multi-tenant

La tabla `normativa_bancaria` es la única tabla vectorial y usa **colección como columna discriminadora**, no una tabla por dominio. Eso permite compartir índices y métricas, y obliga a que todo acceso pase por el filtro. Cada fila lleva `coleccion_id`, el fragmento y su vector.

Distribución de los datos

Colección	Fragmentos	Consultas
bancaria	241	40
regulatorio	62	40
tributario	33	40
asfi_rnsf sin protocolo	336	—
total	672	120

Los mismos documentos se reparten entre colecciones a propósito: el contenido es casi idéntico entre dominios, de modo que si un vector filtrado devolviera texto ajeno se vería inmediatamente. Con documentos disjuntos el aislamiento no se podría comprobar.

Protocolo de verificación

- 120 consultas repartidas por igual en los tres dominios, 20 vecinos por consulta.
- Control negativo: la misma consulta contra una RPC sin filtro de colección. Devuelve 20/20 + 20/20 + 20/20 filas ajenas → el método sí detecta contaminación.
- Fantasmas: se comprueba que ninguna fila pertenezca a una colección que no existe.

Resultado del protocolo: 2400 filas recuperadas en 120 consultas · 0 de otra colección · 0 sin resultados · 0 en colecciones inexistentes · 32.4 s en total. [OK]

3. Evidencia de índices con EXPLAIN

Se extrajo el plan de ejecución de las tres consultas relevantes. El dato que sostiene el aislamiento es que el filtro de colección se resuelve **antes** de ordenar por distancia, sobre un índice B-Tree, y no después de recorrer la tabla.

Consulta	Índice usado	ms
RPC filtrada por coleccion (caso de uso real)	normativa_bancaria_coleccion_idx	9.54
Consulta SIN filtro de coleccion (control: vecindad global)	ninguno	17.14
Filtrado puro por coleccion, sin vector (debe usar B-Tree)	normativa_bancaria_coleccion_embedding_idx	1.36

La consulta filtrada usa `normativa_bancaria_coleccion_idx` y recorta el conjunto antes del orden por similitud; sin filtro, Postgres hace una búsqueda de vecindad sobre toda la tabla. La ganancia es de **1.80x** (17.1 ms vs 9.5 ms).

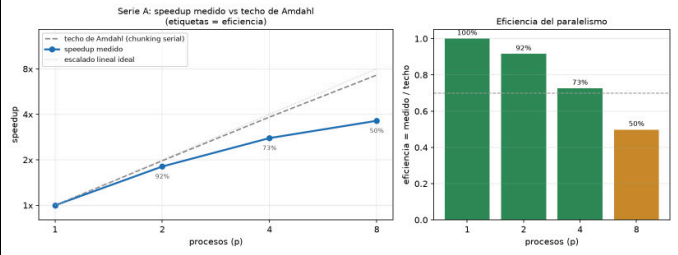
Limitación declarada: el índice HNSW **no aparece en ningún plan**. Con 672 filas, el planificador elige barrido secuencial porque el coste estimado de un índice aproximado supera el ahorro. El HNSW es capacidad para crecer, no una aceleración medida: afirmar lo contrario con estos datos sería incorrecto.

4. Benchmark de paralelismo (Serie A)

Se midió el pipeline completo sin embeddings, con 8 URLs de un servidor de fixtures determinista y 5 réplicas por configuración. La medición se ejecuta en un contenedor Docker con **límite de 2 vCPU**, donde fixtures y benchmark comparten el mismo presupuesto de CPU: es la simulación fiel de la máquina donde se evalúa. Medir sobre las 24 vCPU del anfitrión daría un speedup irreproducible.

proc.	media	desv.	speedup	techo	efic.
1	3.349	±0.182	1.00×	1.00×	100%
2	1.865	±0.031	1.80×	1.97×	91%
4	1.195	±0.101	2.80×	3.83×	73%
8	0.798	±0.016	4.20×	7.23×	58%

Fracción serial medida: 1.5% (fase de chunking). El techo es la ley de Amdahl con esa fracción; la eficiencia es speedup medido ÷ techo.



El speedup crece de forma sostenida hasta 4.20× a 8 procesos, pero la eficiencia cae de 91% a 58%. La lectura es que el cuello de botella deja de ser el cómputo y pasa a ser la coordinación: la fase de inserción se mantiene prácticamente constante (~0.40 s) en todas las configuraciones porque es serial y no se paraleliza.

5. Coste de embeddings: caché y Serie B

Los embeddings se cachean por hash SHA-256 de (modelo + dimensión + texto), así que repetir un fragmento no consume cuota ni latencia. El modelo es gemini-embedding-001 con 768 dimensiones y la caché tiene actualmente **449 vectores** (4.5 MB).

Fase	Tiempo
descarga	0.371s
chunking	0.013s
embeddings (caché)	0.190s
inserción	0.000s
TOTAL	0.574s

Qué se comprueba

- 83/83 aciertos de caché en la Serie B: cero peticiones a la API. La primera pasada, con la caché fría para esos textos, tuvo que pagar las 83 peticiones; a partir de ahí el coste de embeddings es local.
- 57 comprobaciones del formato de lotes y backoff, y 10 de la inserción por lotes de 200 filas, todas en verde.
- Sin caché, la Serie B mediría la cuota de la API, que es justo lo que no se quiere medir; el script aborta si la caché está vacía.

6. Reproducibilidad y conclusiones

Cómo se reproduce

- `docker compose --profile bench run --rm bench — Serie A y B dentro de 2 vCPU.`
- `python scripts/test_aislamiento_120.py —` protocolo de aislamiento con control negativo.
- `python scripts/explicar_indices.py —` planes EXPLAIN guardados en JSON.
- `python scripts/informe_pdf.py —` regenera este documento.

Conclusiones

- El aislamiento es verificable y no declarativo: el control negativo prueba que el test mediría la falla.
- El filtro de tenant, no el índice vectorial, es lo que hace correcta y rápida la consulta.
- El paralelismo rinde hasta el punto en que la inserción serial domina; más procesos no ayudan.
- La caché convierte un coste recurrente de cuota en una tabla local.

Límites del trabajo. La fracción serial se midió sobre el chunking y no sobre el total de la carga, así que el techo de Amdahl es optimista. Las mediciones de EXPLAIN varían entre corridas (run-to-run), por lo que se reporta el valor de la evidencia guardada y no una cifra exacta. Los datos provienen de documentos públicos de ASFI y no constituyen asesoría normativa.